

Deep Networks

From Shallow to Feedforward, and Universal Approximation

Prof. Nirav Bhatt

5 August

Outline

Composing Networks

Where the shallow network left us

- A shallow network is a linear read-out over **learned features**
 $h_d = a[\theta_{d0} + \theta_{d1}x]$ (Classes 5–6)
- With enough hidden units it is a **universal approximator** — but the number of units may be enormous
- Question for today: what if we feed the output of one network into the **input of another**?

Stacking shallow networks gives a **deep network**. The same building block, composed — and far more efficient.

An intuition: solve a big problem in stages

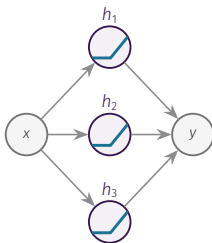
- A shallow network must map raw inputs to the answer in **one leap** — every hidden unit works directly on x
- A deep network solves the problem in **stages**: layer 1 extracts simple features, layer 2 combines them into richer ones, and so on
- Reading is the analogy: **strokes** $\rightarrow$ **letters** $\rightarrow$ **words** $\rightarrow$ **meaning**. Nobody jumps from ink to meaning in a single step
- Each stage is *easy* because it builds on the stage before — the difficulty is **spread across depth**

SHALLOW one leap: raw input $\rightarrow$ answer

DEEP staged: input $\rightarrow$ features $\rightarrow$ features of features $\rightarrow$ answer

Two networks to work with

Network 1



$$h_1 = \text{ReLU}(x + 1)$$

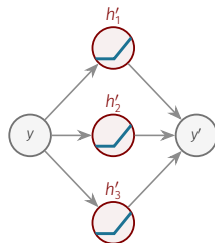
$$h_2 = \text{ReLU}(x + 0.5)$$

$$h_3 = \text{ReLU}(x - 0.5)$$

$$f_1(x) = -1 + 4h_1 - 6h_2 + 6h_3$$

f_1 traces $-1 \rightarrow +1 \rightarrow -1 \rightarrow +1$: three monotone branches.

Network 2



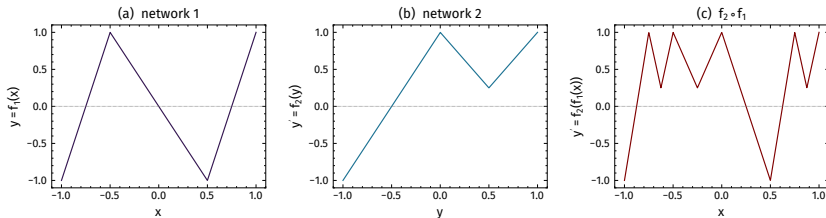
$$h'_1 = \text{ReLU}(y + 1)$$

$$h'_2 = \text{ReLU}(y)$$

$$h'_3 = \text{ReLU}(y - 0.5)$$

$$f_2(y) = -1 + 2h'_1 - 3.5h'_2 + 3h'_3$$

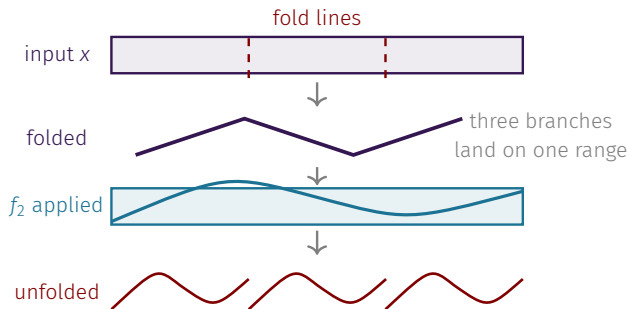
Composing the two networks



(a) f_1 , a zigzag with three branches. (b) f_2 , evaluated over the range f_1 produces. (c) the composition $f_2 \circ f_1$.

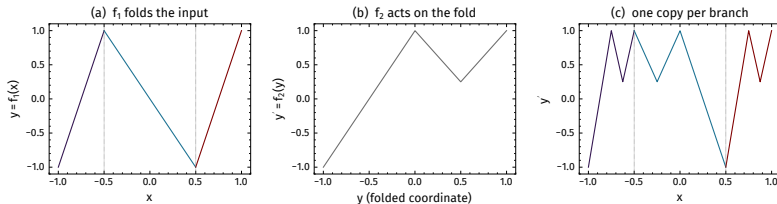
- Each network alone is piecewise linear with a few joints
- The composition has **many more** joints — f_2 's shape appears **once per branch** of f_1

Folding: why the pattern repeats



- f_1 **folds** the input line: three different x land on the same y
- f_2 cannot tell them apart — so its shape is stamped **once per branch** cf. Prince (2023), Fig. 4.3

The same thing, in the plots



(a) the three branches of f_1 , coloured. (b) f_2 on the folded coordinate. (c) the composition — each branch carries its own copy of f_2 , in the matching colour.

SHALLOW joints add: D units $\rightarrow D+1$ regions

DEEP folding *multiplies* regions across layers

Why folding multiplies regions

Counting for the two networks above:

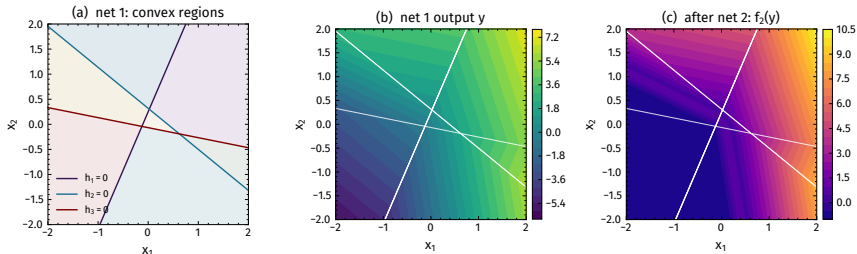
- f_1 has 3 hidden units $\Rightarrow$ 3 branches on $[-1, 1]$
- f_2 has 3 hidden units $\Rightarrow$ 4 linear pieces on its input range
- The composition has up to $3 \times 4 = 12$ linear pieces — from only 6 hidden units

In general, for K layers of D units on a scalar input:

$$\# \text{regions (shallow, } KD \text{ units)} = KD + 1 \quad \text{vs.} \quad \# \text{regions (deep)} \leq (D + 1)^K$$

Width adds regions. Depth multiplies them.

Composing in two dimensions

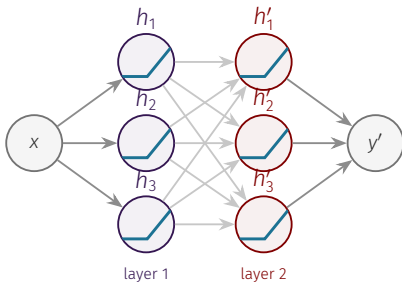


Network 1 here takes $\mathbf{x} = (x_1, x_2)$ into three ReLU units, then one output. (a) each unit contributes a line $h_d = 0$; together they cut the plane into *convex* regions, one per activation pattern. (b) network 1's output is planar on each region. (c) network 2 then bends the values *within* every region — the regions of (a) are subdivided further.

- In 2-D the fold becomes a **crease pattern**; depth creases an already-creased plane

The Deen Network

Two hidden layers



Each circle carries its activation function; arrows are weights.

Layer 1, layer 2, then the output:

$$h_1 = a[\theta_0 + \theta_1 x]$$

$$h_2 = a[\psi_0 + \psi_1 h_1]$$

$$y = \phi_0 + \phi_1 h_2$$

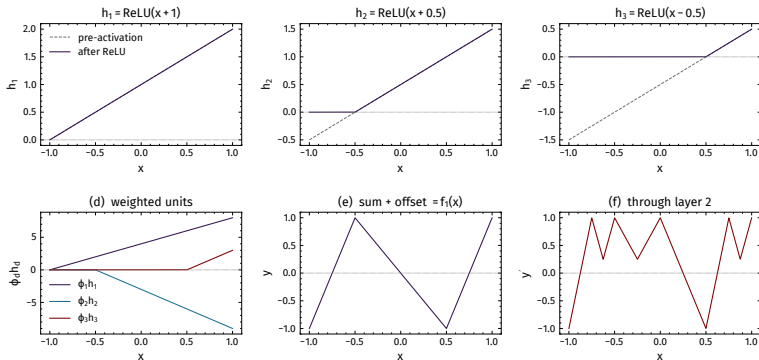
Each layer: **linear map** then **activation**.

Our $f_2 \circ f_1$ is exactly this, with $a = \text{ReLU}$.

SHALLOW one hidden layer

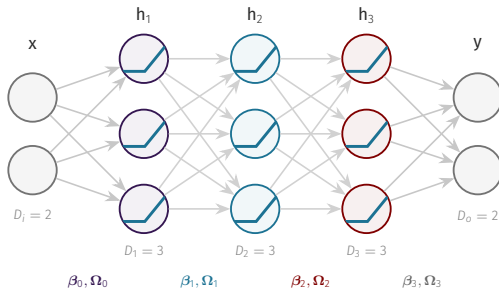
DEEP hidden layers *stacked*: same block, composed

Computed step by step



Top row (a–c): each hidden unit of layer 1 — dashed is the pre-activation $\theta_{d0} + \theta_{d1}x$, solid is after the ReLU clips it at zero. (d) the same units scaled by their output weights ϕ_d . (e) their sum plus the offset: this is f_1 . (f) that signal pushed through layer 2.

The general deep network



$$\mathbf{h}_k = \mathbf{a}[\beta_{k-1} + \Omega_{k-1} \mathbf{h}_{k-1}]$$

- Ω_{k-1} weight matrix, β_{k-1} bias, $\mathbf{a}[\cdot]$ element-wise activation
- **Depth** = hidden layers ($K=3$); **width** = units per layer — both **hyperparameters**
- Parameters here: $9 + 24 + 8 = 41$

Building depth recursively

The two-layer construction **extends recursively** (Bishop & Bishop, 2024, §6.2):

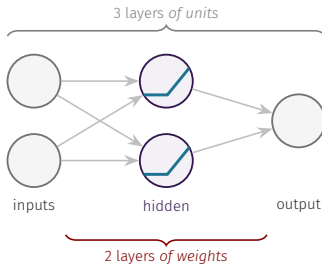
$$\mathbf{z}^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x}), \quad \mathbf{z}^{(2)} = h(\mathbf{W}^{(2)}\mathbf{z}^{(1)}), \quad \dots, \quad \mathbf{y} = f(\mathbf{W}^{(L)}\mathbf{z}^{(L-1)})$$

- Each layer feeds the next — the **same operation** (linear map + nonlinearity) repeated
- The construction extends into a **hierarchical model with many layers** — the basis for deep networks Bishop & Bishop, §6.2
- Nothing new is needed — depth is **repetition of a single idea**

SHALLOW $\mathbf{y} = \phi_0 + \phi h(\mathbf{W}\mathbf{x})$

DEEP $\mathbf{y} = f(\mathbf{W}^{(L)}h(\dots h(\mathbf{W}^{(1)}\mathbf{x})))$

A note on counting layers



- The same network is called “three-layer”, “single-hidden-layer”, or “two-layer” depending on what is counted
- Bishop recommends counting **layers of weights** — the adaptive parameters
Bishop & Bishop (2024), §6.3
- In this course: **deep** = more than one hidden layer

What Deep Networks

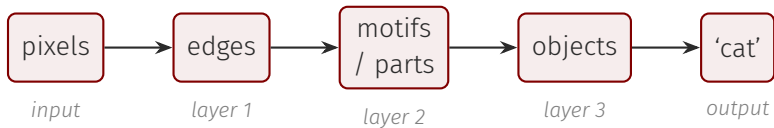
Depth as an inductive bias

- Universality tells us *what* a network *can* represent; it says nothing about what it *prefers* to learn
- A deep architecture encodes an **inductive bias**: that the output relates to the input through a **hierarchy** of intermediate concepts Bishop & Bishop, §6.3.1
- For images, the map from pixels to a concept like ‘cat’ is **highly nonlinear** — hopeless for one layer, natural for many
- The right bias means the network needs **less data** and generalises better

SHALLOW bias: output is one nonlinear map of the input

DEEP bias: output is a **hierarchy** of simpler maps

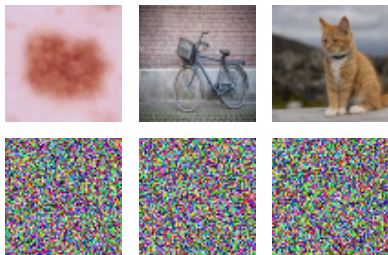
Hierarchical representations



- Early layers detect **low-level** features; later layers assemble them into **high-level** concepts
- Each layer reuses the layer below — features are **shared** by everything built on top

Bishop & Bishop, §6.3.1

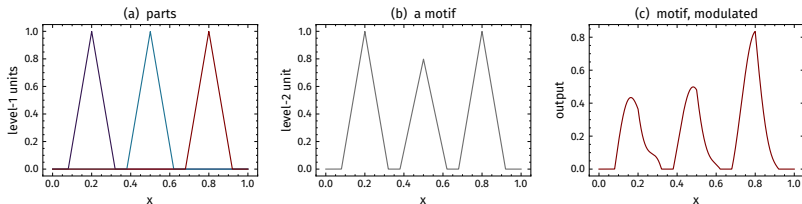
Why hierarchy is the right bias for real data



Bishop & Bishop (2024), Fig. 6.8: natural images
(top) vs. random pixels (bottom).

- Almost every random pixel pattern is **noise** — images like the ‘cat’ above are vanishingly rare among all possible images
- Natural data lives on a **low-dimensional manifold** with strong structure
- That structure is **compositional**, so a compositional model captures it efficiently

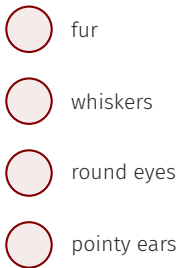
A hierarchy, built explicitly



(a) three level-1 units, each a bump $\text{ReLU}(1 - |x - c|/w)$ centred at $c \in \{0.2, 0.5, 0.8\}$ with $w = 0.12$. (b) a level-2 unit sums them into one motif. (c) level 3 modulates that motif by $0.6 + 0.4 \sin(6\pi x)$.

- Each stage is a **simple** operation on the stage below — yet the result is intricate

Distributed representations



a hidden layer

- Each unit signals whether a **feature** is present (high) or absent (low)
 - M units name M features — but their **combinations** can encode up to 2^M distinct patterns
 - Concepts are represented by **patterns of activity**, not single units — a **distributed code**
- Bishop & Bishop, §6.3.2

SHALLOW features fixed and hand-chosen

DEEP features learned, and **shared combinatorially**

Representation learning: the set-up

Two interleaved half-moons, 400 points, Gaussian noise $\sigma = 0.1$; inputs standardised. Network $2 \rightarrow 24 \rightarrow 2 \rightarrow 1$, all ReLU:

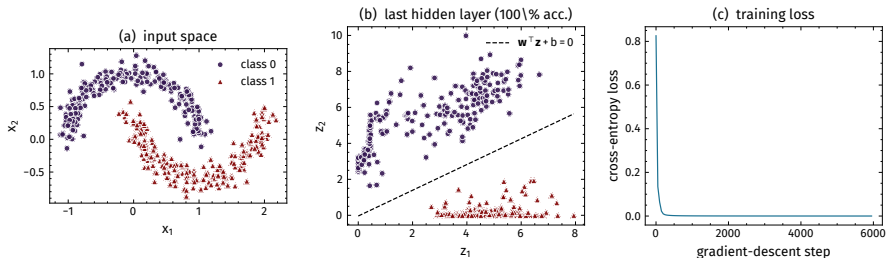
$$\mathbf{z}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1), \quad \mathbf{z}_2 = \text{ReLU}(\mathbf{W}_2 \mathbf{z}_1 + \mathbf{b}_2), \quad s = \mathbf{w}_3^\top \mathbf{z}_2 + b_3$$

Trained by gradient descent on the binary cross-entropy of $p = \sigma(s)$:

$$L = -\frac{1}{N} \sum_n \left[y_n \log p_n + (1 - y_n) \log(1 - p_n) \right], \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L$$

- $\eta = 0.5$, 6000 full-batch steps, He initialisation
- The last hidden layer is **2-dimensional** on purpose — so we can plot the representation the network learns

Representation learning: the result



(a) the data in input space — no straight line separates the classes. (b) the same points plotted in the coordinates (z_1, z_2) of the last hidden layer; the dashed line is the output layer's boundary $\mathbf{w}_3^T \mathbf{z} + b_3 = 0$. (c) the loss over training.

- The network has **rearranged the data** until a line is enough cf. Bishop & Bishop, §6.3.3

What that experiment shows

- Each layer **transforms** the data to make the task easier for the next one
- By the final hidden layer the classes are arranged so a **simple linear classifier** suffices
- That layer's output is the **learned representation** (the *embedding*); the output layer merely draws a line in it
- Nobody designed those coordinates — they were **learned from the data**

Bishop & Bishop, §6.3.3

SHALLOW one transform, then a linear read-out

DEEP many learned transforms, then a linear read-out

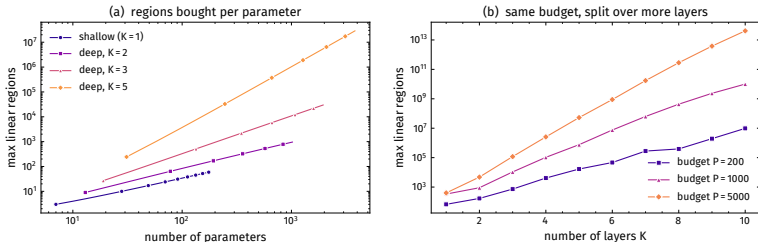
Why Denth?

Universal approximation, revisited

- A shallow network is **already** a universal approximator (Class 5) — so why go deep?
- Because universality says a solution **exists**, not that it is **small**: matching a complex function may need exponentially many hidden units in one layer
- Deep networks can represent the same functions with **far fewer parameters**

The question is not *whether* but *how efficiently* a network represents a function.

Linear regions per parameter



Parameter counts for a 1-input, 1-output ReLU net: $P_{\text{shallow}}(D) = 3D + 1$ and

$P_{\text{deep}}(D, K) = 2D + (K-1)D(D+1) + D + 1$; regions $D+1$ and $(D+1)^K$. (a) regions bought per parameter, log-log.

(b) a fixed budget P spent on K layers: for each K we take the widest D that still fits, then plot the regions it buys.

SHALLOW regions grow *polynomially* with width

DEEP regions grow *exponentially* with depth

Shallow vs. deep: the trade-off

	Shallow (1 layer)	Deep (many layers)
Expressivity	universal, but needs many units	universal with far fewer units
Regions	polynomial in width	exponential in depth
Features	one level of learned features	hierarchical features, layer on layer
Training	convex read-out only	harder landscape; needs backprop (Class 11+)

Depth trades a harder optimisation problem for a dramatic gain in representational efficiency.

Why depth tends to win in practice

- Real data is **compositional**: edges $\rightarrow$ shapes $\rightarrow$ objects; phonemes $\rightarrow$ words $\rightarrow$ meaning
- Deep networks mirror this — each layer builds **features of the previous layer's features**
- Fewer parameters for the same function means **better generalisation** and less data to fit
- Caveat: depth alone is not enough — training deep networks needs the tools of the coming classes (initialisation, normalisation, residual connections)

SHALLOW flat: features of the raw input

DEEP hierarchical: features of features of features

Takeaways

1. A **deep network** is shallow networks **composed**: each layer is a linear map followed by an activation
2. Composition **folds** input space, so regions **multiply** across layers rather than add — $3 \times 4 = 12$ pieces from 6 units
3. Both shallow and deep nets are universal — but deep nets reach the same functions with **exponentially fewer parameters**
4. Depth suits **compositional** data by building hierarchical features, as the half-moons experiment showed

Next: depth and width efficiency theorems, then how to actually **train** these networks (backpropagation).

References

- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Chapter 4 — composition, folding, region counting, deep network formulation.
- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Chapter 6 — recursive construction, layer counting, hierarchical and distributed representations, representation learning. Figure 6.8 reproduced with attribution.
- G. Cybenko (1989); K. Hornik (1991); Montúfar et al. (2014) on linear regions and depth.
- Plots generated with `src/make_figures.py` (NumPy, Matplotlib, SciencePlots).

Questions?